

CURSO DE STUDIO 5000 LOGIX DESIGNER

MÓDULO — LÓGICA DE CONTROL

Fundamentos de la lógica de escalera

Ladder Logic aplicado a PLC Allen-Bradley

**Enfoque:** Automatización Industrial • ControlLogix / CompactLogix • Studio 5000 • Diagnóstico y buenas prácticas de planta**Propósito:** aprender a leer, construir, probar y diagnosticar lógica Ladder conectando sensores, Tags, permissives, interlocks, comandos y actuadores.**Nota técnica:** el comportamiento exacto de tareas, instrucciones, I/O y funciones depende del controlador, firmware y versión de Studio 5000. Verifica siempre la documentación del proyecto real.

1. Objetivos de aprendizaje

Este capítulo construye la base para programar y diagnosticar Ladder en Logix Designer. La meta es comprender qué condición está evaluando el PLC y por qué una salida termina activada o desactivada.

- Explicar qué es Ladder y por qué se utiliza en PLC industriales.
- Comprender bit, estado lógico, rung, instrucción y Tag.
- Distinguir XIC, XIO y OTE y leer su intención lógica.
- Construir condiciones AND, OR y NOT.
- Entender el efecto del ciclo de ejecución sobre la lógica.
- Construir un patrón Start/Stop con enclavamiento.
- Separar permissives, interlocks, faults, comandos y feedbacks.
- Introducir temporizadores, contadores y comparaciones.
- Relacionar Ladder con I/O, instrumentación, PowerFlex y protección de motor.
- Diagnosticar fallas siguiendo la cadena entrada → lógica → comando → dispositivo.

Ruta de aprendizaje



2. Introducción

Ladder Logic (LAD) es un lenguaje de programación para PLCs con una representación visual inspirada en circuitos de control por relés. En Studio 5000, una rutina Ladder contiene rungs que ejecutan instrucciones usando Tags y datos del controlador.

En una planta, Ladder puede controlar motores, válvulas, transportadores, bombas, secuencias de proceso y condiciones de permiso. Su valor para mantenimiento está en poder seguir visualmente la cadena de condiciones que conduce a una acción.



Regla de oro: ante una falla, no preguntes primero “¿qué instrucción está mal?”. Pregunta “¿qué condición dejó de ser verdadera y cuál es su origen físico o lógico?”.

Ladder es una herramienta de control. Las funciones de seguridad de una máquina deben implementarse con la arquitectura y dispositivos de seguridad correspondientes; una condición Ladder ordinaria no debe asumirse como función de seguridad.

3. Fundamentos: bit, Tag y rung

3.1 Bit / BOOL

Un **BOOL** representa un estado binario: verdadero/falso o 1/0. Es la base de muchos estados y comandos digitales.

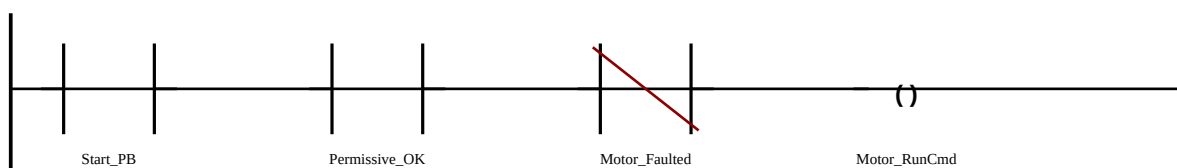
Ejemplos: Start_PB, Stop_PB, Motor_RunFb, Motor_Faulted, Permissive_OK.

3.2 Tag

Un **Tag** es el nombre con el que el programa identifica y manipula un dato. La calidad de la nomenclatura afecta directamente la mantenibilidad.

3.3 Rung

Un **rung** es una línea de lógica. Se analiza conceptualmente de izquierda a derecha. Las condiciones forman el camino lógico y una instrucción de salida puede actuar sobre el resultado.



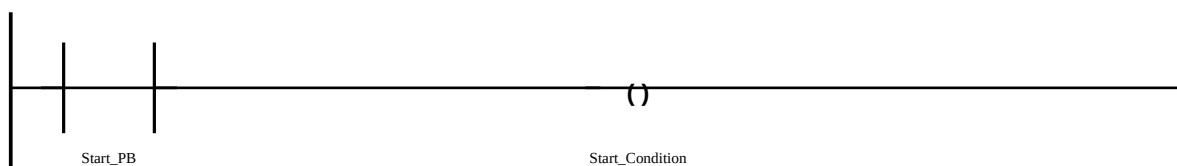
Lectura: **Start_PB AND Permissive_OK AND NOT Motor_Faulted → Motor_RunCmd.**

Error común: confundir el símbolo Ladder con el estado físico de un contacto eléctrico. XIC/XIO son instrucciones que examinan el estado de un Tag; no son una representación literal del contacto físico.

4. Instrucciones fundamentales: XIC, XIO y OTE

4.1 XIC — Examine If Closed

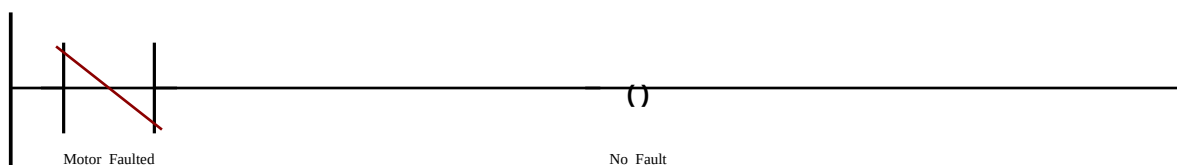
XIC resulta verdadero cuando el bit que examina está en 1. Es común para representar condiciones que deben estar activas.



Start_PB = 1 → XIC verdadero. Start_PB = 0 → XIC falso.

4.2 XIO — Examine If Open

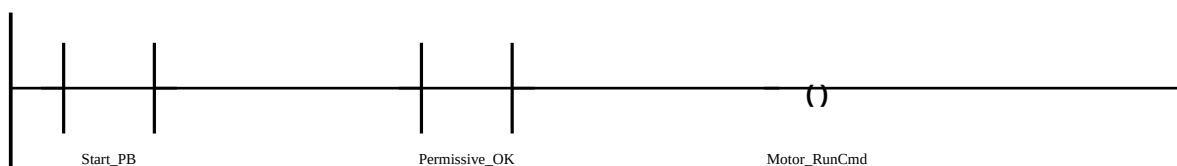
XIO resulta verdadero cuando el bit examinado está en 0. Es frecuente para expresar “no hay falla”, “no hay paro” o una condición que debe permanecer inactiva.



Motor_Faulted = 0 → XIO verdadero. Motor_Faulted = 1 → XIO falso.

4.3 OTE — Output Energize

OTE escribe el resultado lógico de su rung en el Tag de salida. Debes saber quién escribe un Tag y evitar múltiples escritores sin una arquitectura deliberada.

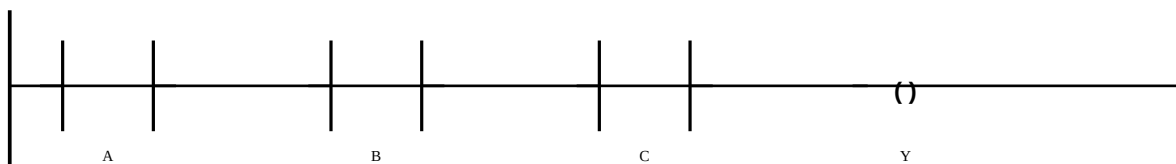


Principio: si Motor_RunCmd no tiene el valor esperado, revisa primero el rung y sus condiciones antes de investigar el motor.

5. Lógica booleana aplicada a Ladder

5.1 Serie = AND

Todas las condiciones en serie deben ser verdaderas.



$Y = A \text{ AND } B \text{ AND } C$

5.2 Paralelo = OR

Una rama paralela permite que cualquiera de los caminos pueda completar la condición.

$(\text{Start_PB OR Auto_Start OR HMI_Start}) \text{ AND Permissive_OK} \rightarrow \text{RunCmd}$

5.3 Negación = NOT

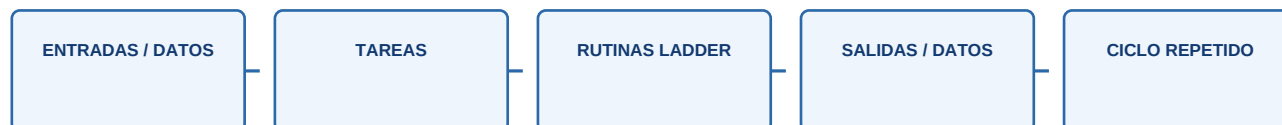
La negación invierte el estado lógico. XIO es una forma común de expresar que un bit debe estar en 0.

Traducción: “si existe una solicitud de arranque, los permisos están OK y no existe una falla, entonces permite el comando”.

Buenas prácticas: escribe primero la ecuación en palabras y después conviértela a Ladder.

6. Ejecución y ciclo de scan

El controlador ejecuta repetidamente su lógica de acuerdo con la configuración de tareas y programas. El modelo simplificado sirve para aprender; la ejecución real puede involucrar tareas continuas, periódicas y de evento, además de I/O y comunicaciones.



El orden de ejecución importa cuando el mismo Tag es escrito desde varios lugares. Un segundo rung puede modificar el valor producido por el primero.

Ejemplo: Rung 1 activa Motor_RunCmd por permisos y Rung 20 lo desactiva por otra condición. Localiza ambos escritores y conoce su orden antes de modificar.

Buenas prácticas: centraliza la decisión de comandos cuando sea posible y usa Cross Reference para encontrar lectores/escritores.

7. Start/Stop y enclavamiento

El patrón Start/Stop enseña cómo una acción momentánea puede mantenerse mediante realimentación lógica. Es un patrón clásico, pero una máquina real debe integrarlo con permissives, interlocks, modos y seguridad.

```
(Start_PB OR Motor_RunCmd) AND NOT Stop_PB AND NOT Motor_Faulted → Motor_RunCmd
```

La rama de Motor_RunCmd mantiene el comando después de soltar Start. Stop o una falla rompen el camino lógico.

Ejemplo de planta: banda transportadora. START solicita movimiento; con permisos correctos se genera RunCmd. El drive puede devolver RunFb.

Error común: usar RunCmd como si fuera RunFb. Un comando significa “el PLC está solicitando”; un feedback significa “el equipo está confirmando”.

8. Permissives, Interlocks y Faults

Permissive: condición necesaria para permitir una operación. Ej.: equipo disponible o condición de proceso correcta.

Interlock: condición que bloquea o limita una operación cuando no debe ejecutarse. La filosofía exacta depende del proceso.

Fault: condición anormal detectada por lógica, I/O o dispositivo. Puede provocar paro, bloqueo o alarma según estrategia.



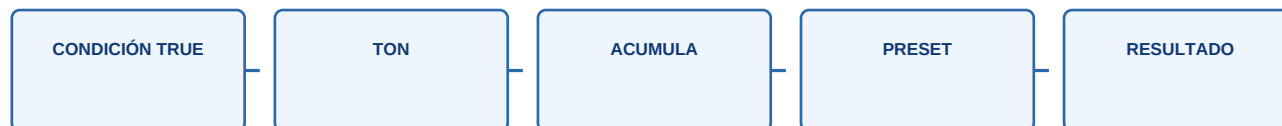
Ejemplo: `RunCmd = StartRequest AND Permissive_OK AND NOT Interlock AND NOT Faulted.`

Importante: un interlock de proceso no equivale automáticamente a una función de seguridad.

Buenas prácticas: permite diagnosticar por qué no arranca: `Permissive_Not_OK`, `Interlock_Active`, `Faulted`, etc.

9. Temporizadores: introducir tiempo

Los temporizadores permiten incorporar tiempo a la lógica. En Logix, TON (Timer On Delay) es una instrucción fundamental.



Ejemplo: después de arrancar una bomba, esperar un intervalo antes de habilitar una etapa posterior. Si el proceso requiere confirmación física, el tiempo no sustituye al feedback.

Troubleshooting: si no llega a Done, observa condición de entrada, preset, acumulado y contexto de ejecución.

Nota: los miembros y comportamiento exactos deben verificarse en la documentación de la versión utilizada.

10. Contadores y comparaciones

Los contadores registran eventos como piezas o ciclos. Las comparaciones convierten valores numéricos en decisiones de control.

```
Product_Detected → Counter → Part_Count >= Batch_Target → Batch_Complete
```

Problema típico: un contador aumenta demasiado porque la entrada permanece TRUE durante varios ciclos. Diseña la detección del evento según el comportamiento de la señal.

Aplicación: una estación de empaque cuenta cajas y finaliza el lote al alcanzar el objetivo.

Precaución: disponibilidad y comportamiento exactos de instrucciones dependen del controlador y versión.

11. Ladder + I/O + instrumentación

Ladder conecta el mundo físico con la decisión de control. Para una entrada digital: sensor → módulo DI → I/O Tag → lógica. Para una señal analógica se agrega interpretación/escalamiento a unidades de ingeniería.



Digital: Prox_ProductPresent : BOOL. **Analógica:** Pressure_PV : REAL, si el diseño y módulo lo justifican.

Diagnóstico DI: revisar estado online → módulo/canal → alimentación/cableado → sensor → configuración → Tag → lógica.

Analógica: no uses una escala raw universal; revisa el módulo y su configuración.

12. Integración conceptual con PowerFlex y E300/E3 Plus

Ladder puede generar comandos para un PowerFlex y procesar estados/feedbacks. El intercambio puede hacerse mediante I/O o EtherNet/IP según el diseño.



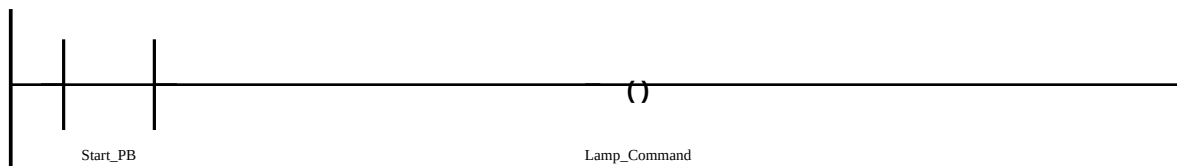
Tags funcionales típicos: Drive_RunCmd : BOOL, Drive_RunFb : BOOL, Drive_Faulted : BOOL, Drive_SpeedRef : REAL, Drive_SpeedFbk : REAL. Son ejemplos de nomenclatura, no nombres obligatorios.

Diagnóstico: RunCmd=1 y RunFb=0 → revisar lógica → permissives → comunicación/I/O → estado del drive → inhibit/enable → falla → motor → mecánica.

E300 vs E3 Plus: no los trates como idénticos. Los datos, comunicación, perfiles y funciones dependen del modelo y configuración.

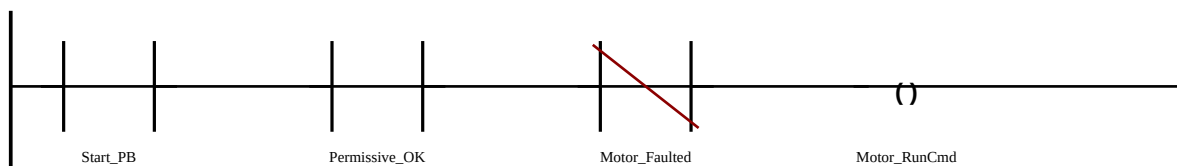
13. Ejemplos prácticos — Básico → Avanzado

Ejemplo 1 — Básico: lámpara



Aprendizaje: BOOL → XIC → OTE.

Ejemplo 2 — Intermedio: motor con permisos



Aprendizaje: AND + NOT + comando final.

Ejemplo 3 — Avanzado: manual/automático

```
(Manual_Start OR Auto_Start OR HMI_Start) AND Permissive_OK AND NOT Interlock AND NOT Faulted → RunCmd
```

Ejemplo 4 — Instrumentación

```
Pressure_PV > HighLimit → Pressure_High_Alarm
```

Aprendizaje: dato analógico convertido en decisión lógica.

14. Ejemplo completo de planta — Banda transportadora

Descripción: banda con sensor de producto, motor con PowerFlex, protección de motor mediante arquitectura E300/E3 Plus según instalación, pulsadores y HMI.

Tags funcionales

Tag	Tipo	Función
Start_PB	BOOL	Solicitud local
Stop_PB	BOOL	Paro local
Auto_Mode	BOOL	Modo automático
Permissive_OK	BOOL	Permisos de proceso
Interlock_Active	BOOL	Bloqueo de proceso
Motor_Faulted	BOOL	Falla reportada
Motor_RunCmd	BOOL	Comando lógico final
Motor_RunFb	BOOL	Confirmación
Speed_Ref	REAL	Referencia
Speed_Fbk	REAL	Feedback
Product_Present	BOOL	Sensor

Secuencia de funcionamiento



Lógica conceptual

```

Request = Start_PB OR Auto_Start OR HMI_Start
RunCmd = Request AND Permissive_OK AND NOT Interlock_Active AND NOT Motor_Faulted AND NOT Stop_PB
  
```

Diagnóstico de arranque

1. Confirmar que la solicitud está activa.
2. Confirmar cada permissive individual.
3. Confirmar que Interlock_Active=0 y Motor_Faulted=0.
4. Confirmar que Motor_RunCmd cambia a 1.
5. Si RunCmd=1 pero RunFb=0, investigar drive, comunicación I/O, estado y motor.
6. Si RunFb=1 pero no hay movimiento, investigar feedback, referencia, motor y mecánica.

Principio profesional: separar solicitud, comando y confirmación hace que el sistema sea mucho más fácil de mantener y diagnosticar.

15. Tabla de troubleshooting

Síntoma	Posibles causas	Cómo comprobar	Acción recomendada
Motor no arranca	Start; permiso; interlock; fault	Monitorear Tags del rung	Encontrar primera condición falsa
RunCmd=1, RunFb=0	Drive; comunicación; inhibit; fault; motor	Revisar estado y ruta	Aislar software/comunicación/equipo
RunCmd cambia y vuelve a 0	Otro rung escribe el Tag	Cross Reference + monitoreo	Encontrar segundo escritor
Timer no termina	Entrada cae; preset/configuración	Observar condición y acumulado	Diagnosticar causa de entrada
Contador cuenta de más	Entrada permanece TRUE	Observar señal por varios ciclos	Diseñar detección de evento
DI no cambia	Sensor/cableado/módulo/configuración	I/O online + medición	Separar campo/I/O/lógica
HMI muestra valor incorrecto	Tag equivocado; lógica sobrescribe	Tag + Cross Reference	Corregir fuente de verdad
Proceso falla con lógica correcta	Feedback; drive; motor; mecánica	Comparar comando/feedback/físico	Continuar diagnóstico por capas

16. Caso de troubleshooting — resuélvelo antes de mirar la guía

Síntoma: la banda no arranca desde el pulsador local. La HMI muestra "Ready".

```
Valores observados:  
Start_PB = 1  
Stop_PB = 0  
Motor_Faulted = 0  
Motor_RunCmd = 0  
Motor_RunFb = 0
```

Tu tarea: indica qué revisarías primero, segundo y tercero. ¿Qué Tag adicional necesitas? ¿Qué esperarías observar en el rung? ¿Cuándo pasarías de software a comunicación/drive?

No cambies el programa todavía. El objetivo es aislar la primera condición que rompe la cadena.

17. Guía de solución del caso

1. Revisar el rung que genera Motor_RunCmd. Los valores dados ya descartan Start, Stop y Faulted como bloqueos directos.
2. Revisar Permissive_OK y, si existe, cada permissive individual. "Ready" de HMI no sustituye la comprobación del PLC.
3. Revisar Interlock_Active y cualquier condición adicional del rung. Usar Cross Reference si es necesario.
4. Confirmar si Motor_RunCmd está siendo escrito por otra rutina/rung y localizar todos los escritores.
5. Si la lógica no produce RunCmd, el problema está probablemente en programación/condiciones. Si RunCmd pasa a 1 y RunFb permanece 0, pasar a I/O/EtherNet/IP, drive/protección, motor y mecánica.
6. Después de corregir, probar de forma controlada, verificar feedback y documentar el cambio.

Razonamiento: el diagnóstico profesional avanza desde la condición lógica cercana al síntoma hacia el campo. No se reemplaza un motor cuando todavía no se demostró que el PLC genera el comando.

18. Buenas prácticas de ingeniería

Diseña antes de programar

Escribe la secuencia y condiciones en lenguaje claro antes de dibujar Ladder.

Nombres funcionales

Usa nombres como Motor01_RunCmd, Motor01_RunFb y Motor01_Faulted.

Una fuente de verdad

Evita múltiples escritores ambiguos para el mismo comando.

Permissives visibles

Haz que el diagnóstico muestre qué permiso falta.

Comandos vs feedbacks

Nunca asumas que RunCmd significa movimiento real.

I/O separado de proceso

Cuando el estándar lo permita, usa Tags funcionales para desacoplar lógica y hardware.

Comentarios útiles

Explica intención y condiciones especiales.

Prueba por etapas

Prueba entrada, lógica, comando, feedback y actuador por separado.

Online con disciplina

Antes de escribir/forzar, evalúa impacto, autorización y reversión.

Documenta cambios

Registra qué cambió, por qué, cómo se probó y cómo revertirlo.

19. Ejercicios progresivos

Resuélvelos primero. Las soluciones se encuentran después.

Nivel básico — 5

1. Define Tag, BOOL, rung y condición lógica con un ejemplo.
2. Si Start_PB=1, ¿qué resultado produce XIC Start_PB?
3. Si Faulted=1, ¿qué resultado produce XIO Faulted?
4. Traduce: $A \text{ AND } B \text{ AND NOT } C \rightarrow \text{MotorCmd}$.
5. Explica la diferencia entre comando y feedback.

Nivel intermedio — 5

1. Diseña Tags para Start, Stop, Permissive, Fault, RunCmd y RunFb de un motor.
2. Escribe la ecuación para arrancar con Start o HMI_Start, con Permissive_OK y No_Fault.
3. Explica por qué un contador puede aumentar más de una vez por pieza.
4. Propón tres pruebas para determinar por qué un DI no cambia en Studio 5000.
5. Explica por qué múltiples OTE sobre un Tag complican el diagnóstico.

Nivel avanzado — 3

1. Diseña lógica de motor con Manual_Start, Auto_Start, Permissive_OK, Interlock_Active, Faulted, RunCmd y RunFb.
2. Diseña una estrategia para que HMI solicite velocidad, con modo, permisos y límites.
3. Describe cómo diagnosticar RunCmd=1, RunFb=0 sin cambiar parámetros del drive.

Troubleshooting

Una bomba no arranca. Start_PB=1, pero Permissive_OK=0. Enumera la cadena de diagnóstico para descubrir cuál permiso está bloqueando.

20. Soluciones explicadas

1. Tag = nombre del dato; BOOL = dato binario; rung = línea de lógica; condición = expresión que puede resultar verdadera o falsa.
2. XIC con Start_PB=1 → verdadero.
3. XIO con Faulted=1 → falso.
4. MotorCmd se activa cuando A=1, B=1 y C=0.
5. RunCmd es solicitud; RunFb es confirmación del equipo.
6. Separar comandos, estados, fallas y permisos mejora trazabilidad y mantenimiento.
7. Request = Start_PB OR HMI_Start; RunCmd = Request AND Permissive_OK AND NOT Faulted.
8. Porque la entrada puede permanecer TRUE durante varios scans; el diseño debe detectar el evento adecuadamente.
9. Revisar estado I/O online → canal/módulo → alimentación/cableado → sensor → configuración → Tag → lógica.
10. Porque el valor final puede depender del orden de ejecución y del último escritor; usar Cross Reference.
11. Separar solicitud, permisos, interlocks y falla; producir RunCmd; usar RunFb para confirmación.
12. Definir límites, modo, permisos, unidades y fuente de verdad antes de permitir escritura desde HMI.
13. Si RunCmd=1 y RunFb=0: revisar comunicación/I/O, estado del drive, inhibit/enable, fallas, referencia, motor y mecánica.
14. Desglosar Permissive_OK en permisos individuales y encontrar el primero falso; seguirlo hasta su origen.

21. Evaluación final

Realízala sin consultar las soluciones. Evalúa comprensión, análisis y aplicación.

10 preguntas teóricas

1. ¿Qué es Ladder Logic y qué ventaja ofrece en mantenimiento?
2. ¿Qué representa un BOOL?
3. ¿Qué hace XIC?
4. ¿Qué hace XIO?
5. ¿Qué hace OTE?
6. ¿Qué diferencia hay entre serie y paralelo?
7. ¿Qué es un permissive?
8. ¿Qué es un interlock?
9. ¿Qué diferencia hay entre RunCmd y RunFb?
10. ¿Por qué importa el orden de ejecución?

5 preguntas de análisis

1. Un rung tiene condiciones visualmente verdaderas pero la salida no cambia. ¿Qué investigarías?
2. Un motor arranca desde HMI pero no desde pulsador. ¿Qué compararías?
3. Una DI aparece activa en el módulo pero el proceso no responde. ¿Qué capa analizarías?
4. RunCmd cambia a 1 y vuelve a 0. ¿Cómo localizarías al escritor que lo apaga?
5. Una presión analógica genera una alarma demasiado pronto. ¿Qué verificarías antes de cambiar el límite?

3 problemas prácticos

1. Diseña la lógica conceptual de una banda: Start/Stop + permissives + fault + RunCmd + RunFb.
2. Diseña una secuencia: bomba → espera de tiempo → válvula, usando feedback cuando sea necesario.
3. Una máquina tiene Manual y Auto. Define solicitudes, condiciones comunes y comando final.

2 problemas de troubleshooting

1. RunCmd=0 con Start=1. Explica tu procedimiento completo.
2. RunCmd=1, RunFb=0 y drive en estado normal. Explica qué capas revisarías.

Problema industrial integrador

Una banda con PowerFlex no arranca. Hay sensor de producto, HMI, permissives, interlocks y protección de motor. Diseña Tags y procedimiento de diagnóstico desde pulsador hasta motor.

22. Resumen técnico — hoja de estudio



Qué recordar:

- Ladder representa condiciones de control de forma visual.
- XIC: verdadero cuando el bit examinado está en 1.
- XIO: verdadero cuando el bit examinado está en 0.
- OTE: escribe el resultado del rung en un Tag.
- Serie = AND; paralelo = OR; inversión = NOT.
- RunCmd ≠ RunFb: intención no es confirmación.
- Permissive permite; Interlock bloquea; Fault indica una condición anormal.
- El scan y el orden de ejecución importan, especialmente con múltiples escritores.
- Timers y counters agregan tiempo y conteo a la lógica.
- Para diagnosticar: entrada → Tag → condición → rung → comando → comunicación/I/O → dispositivo → proceso.
- Una solución profesional identifica la causa raíz y deja el sistema más diagnosticable.

Mapa mental final



Checklist de ingeniero: ¿Qué debería ocurrir? ¿Qué Tag representa la condición? ¿Quién lo escribe? ¿Quién lo lee? ¿Cuál es la primera condición falsa? ¿Qué prueba confirma la causa? ¿Cómo verifico la reparación?

Fin del capítulo — Fundamentos de la lógica de escalera